- Assert Syntax
- Mock Objects
- Define/Use Paths
- Slice-Based Testing
- Path Graph / Control Flow Graph (You may be given code and asked to draw the graph

Mock Objects

- Despite the name, some units require the presence of other units, either as sources of inputs or as destinations of outputs (messages).
- In Chapter 13, we will see the need for “Stubs” and “Drivers” for procedural code integration testing.
- Mock objects take the place of stubs and drivers for o-o software.
- See example in Listing 9.4, repeated on the next slide.
- Notice that the override acts as a driver for procedural code..
  - It contains the assertion inputs (2,29,2000) and (2,29,2001)
  - the expected outputs return true or return false

```
@Test
public void testWithoutDependencyManual() {
    // This test has no dependency on SimpleDate.isLeap
    SimpleDate simpleDate = new SimpleDate(1, 1, 2000) {
        @Override
        protected boolean isLeap(int year) {
            if(2000 == year)
                return true;
            else if(2001 == year)
                return false;
            else
                throw new IllegalArgumentException("No Mock for year " + year);
        }
    };

    assertTrue(simpleDate.rangesOK(2, 29, 2000)); // Valid due to leap year
    assertFalse(simpleDate.rangesOK(2, 29, 2001)); // Valid due to leap year
}
```

Assert Syntax

Assert Syntax

- Can assert True or False
- Refer to Class.Method
- Can pass values to the method being tested.
- assertTrue(simpleDate.rangesOK(2, 29, 2000));
- assertFalse(simpleDate.rangesOK(2, 29, 2001));

Listing 9.2….

```
import org.junit.Test;
import static org.junit.Assert.*;

public class SimpleDateTest {

    @Test
    public void testWithDependency() {
        // This test has a dependency on SimpleDate.isLeap
        SimpleDate simpleDate = new SimpleDate(1, 1, 2000);

        assertTrue(simpleDate.rangesOK(2, 29, 2000)); // Valid due to leap year
        assertFalse(simpleDate.rangesOK(2, 29, 2001)); // Invalid due to leap year
    }
}
```

Slice-based Testing

- The concept of program slices was first proposed in 1979.
- Program slices mimic the way programmers, and especially maintenance programmers, think.
- Slices focus on portions of code that are of interest to some question.
- We continue with the same context we had for Define/Use Testing:
- A set V of variables in a program P with program graph G(P). Usually the set V contains a single variable, v)
- There are four distinctions among slices..
  - Forward slices
  - Backward slices
  - Static slices
  - Dynamic slices

Slice Definitions

- Given a program P and a set V of variables in P, a ***backward slice on the variable set V at statement n***, written S(V, n), is the set of all statement fragments in P that contribute to the values of variables in V at node n.

- Given a program P and a set V of variables in P, a ***forward slice on the variable set V at statement n***, written S(V, n), is the set of all statement fragments in P that are affected by the values of variables in V at node n.